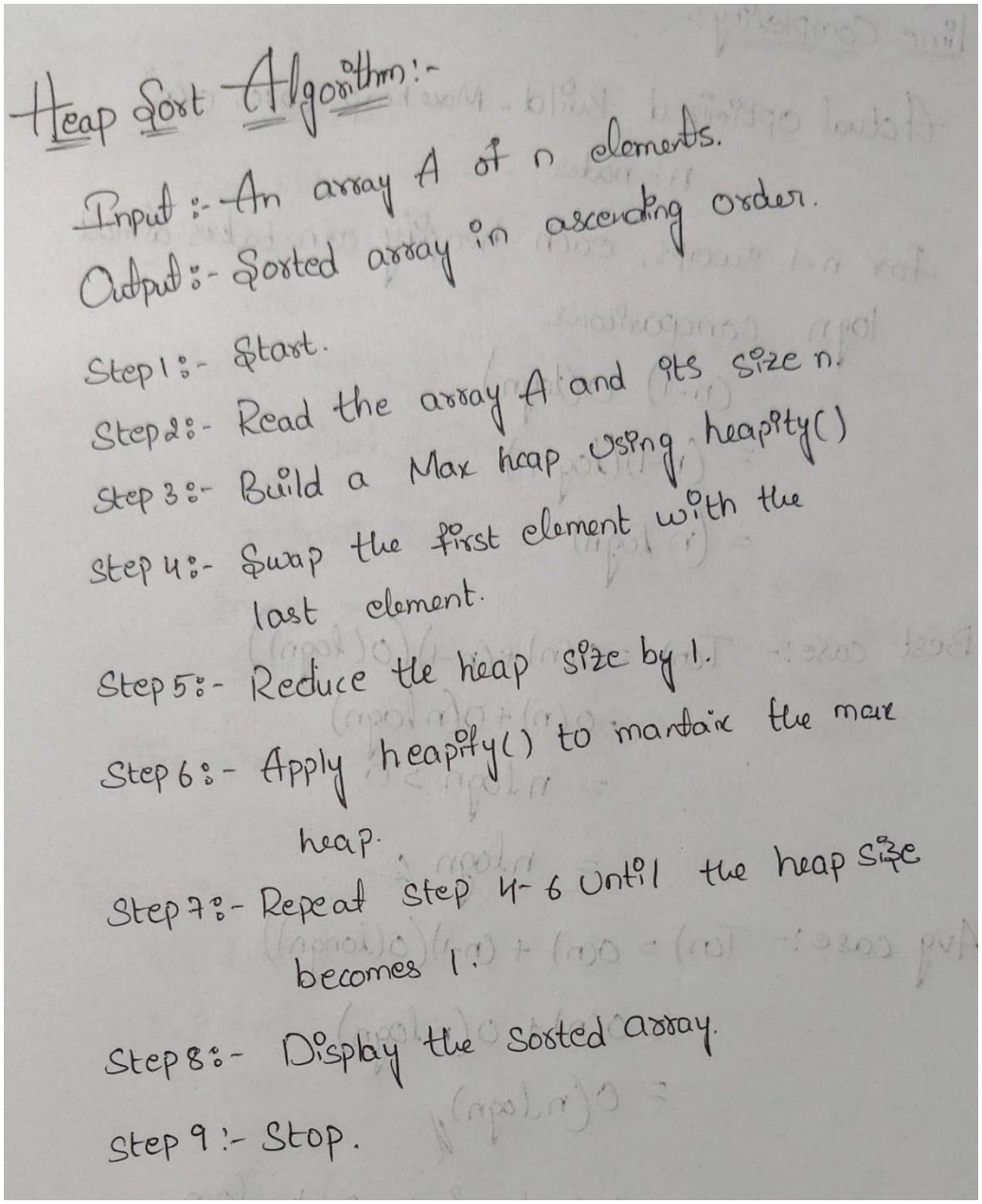
Practical-3

Aim:

To implement Heap Sort using the Heapify technique in C++ and analyze its time complexity

Algorithm:



Heap Sort Algorithm:-

Input :- An array A of n elements.
Output :- Sorted array in ascending order.

Step 1 :- Start.
Step 2 :- Read the array A and its size n.
Step 3 :- Build a Max heap using heapify()
Step 4 :- Swap the first element with the last element.
Step 5 :- Reduce the heap size by 1.
Step 6 :- Apply heapify() to mantain the max heap.
Step 7 :- Repeat step 4-6 until the heap size becomes 1.
Step 8 :- Display the sorted array.
Step 9 :- Stop.

Program:

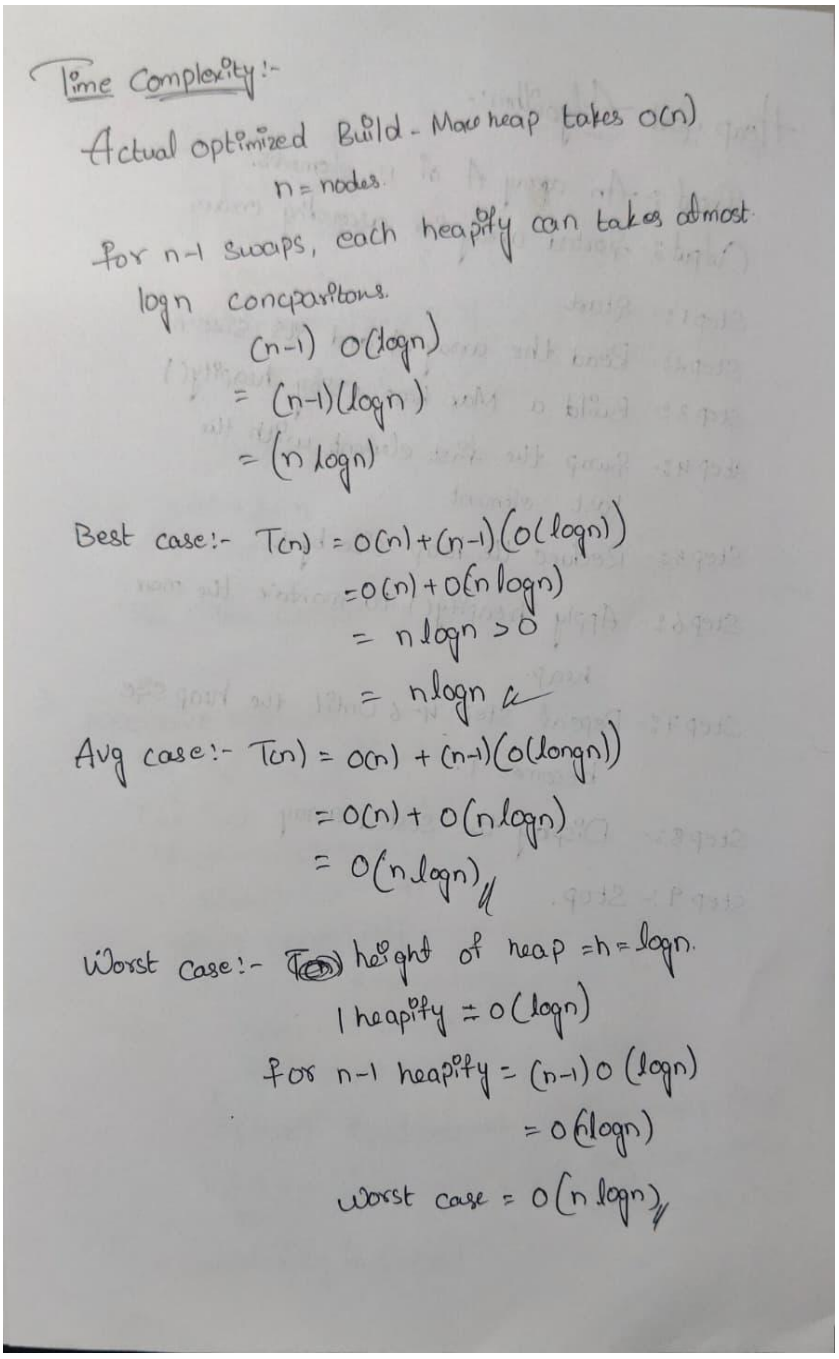
```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i) {
   int largest = i;
   int left = 2 * i ;
   int right = (2 * i) + 1;

   if (left < n && arr[left] > arr[largest])
      largest = left;

   if (right < n && arr[right] > arr[largest])
      largest = right;

   if (largest != i) {
      swap(arr[i], arr[largest]);

      heapify(arr, n, largest);
   }
}
void heapSort(int arr[], int n) {
   for (int i = n / 2 - 1; i >= 0; i--)
      heapify(arr, n, i);
```

```
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);

        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";
    printArray(arr, n);
    heapSort(arr, n);

    cout << "Sorted array:   ";
    printArray(arr, n);
    return 0;
}
```

Output:

```
Original array: 12 11 13 5 6 7
Sorted array:   5 6 7 11 12 13
```

Time Complexity:



Conclusion:

Heap Sort was successfully implemented using the Heapify technique. The array was sorted in ascending order with O(n log n) time complexity.